CODERBLUEPRINT

JAVA • SPRING BOOT • MICROSERVICES

100 Most Asked Interview Questions & Answers

Interview-focused preparation for Java Backend roles

Table of Contents

1. Java Memory Management
2. Java Streams API
3. Java 8 Features
4. Comparable vs Comparator
5. Collections Framework, HashMap Internals & equals/hashCode
6. LFU Cache Implementation
7. Fail-Fast vs Fail-Safe Collections
8. Thread-Safe Collections
9. Coding Questions - Strings, Sorting, HashMap, LinkedList
10. ArrayList vs LinkedList
11. Cloning and Copy Constructor
12. Design Patterns - Singleton, Builder, Chain of Responsibility, Adapter
13. Spring Boot - Annotations, REST, Exception Handling & @Transactional
14. JPA, Hibernate & Database Locking
15. Asynchronous Processing in Spring (@Async)
16. ExecutorService - Deep Dive
17. Microservices - Types, Communication, Circuit Breaker & JWT
18. Spring Security
19. AWS Services (commonly asked for Java/Spring Boot roles)
20. Apache Kafka - Topics, Producers, Consumer Groups & Idempotency
21. General / Frequently Asked Core Java Questions

Guide focus: The questions are selected and organized for interview preparation, with answers covering the concepts, internals, trade-offs, common pitfalls, and practical code where relevant.

1. Java Memory Management

Q1. How does memory management work in Java (JVM Memory Model)?

Java memory is managed automatically by the JVM through the Garbage Collector, so developers rarely allocate or free memory manually. When the JVM starts, it reserves a block of memory from the OS and divides it into several runtime areas.

- Heap - stores all objects and arrays; shared across threads; managed by the Garbage Collector.
- Stack - one per thread; stores method call frames, local variables, and partial results; entries are pushed/popped as methods are called and return.
- Method Area / Metaspace (Java 8+) - stores class metadata, static variables, constant pool, and method bytecode. Metaspace lives in native (off-heap) memory, not the JVM heap.
- PC Register - holds the address of the currently executing instruction, per thread.
- Native Method Stack - supports native (JNI) method calls.

When an object has no more reachable references, it becomes eligible for Garbage Collection, and the GC reclaims that memory in the background.

Q2. What is Heap memory in Java?

Heap is the runtime data area where all Java objects and arrays are allocated. It is created when the JVM starts and is shared by all threads of the application. Its size can be tuned using -Xms (initial size) and -Xmx (maximum size).

- Young Generation - split into Eden and two Survivor spaces (S0, S1); new objects are created here; minor GC runs frequently and is fast.
- Old (Tenured) Generation - objects that survive multiple minor GC cycles are promoted here; major/full GC runs less often but is more expensive.
- (Pre-Java 8) PermGen used to store class metadata; replaced by Metaspace in Java 8 which grows in native memory instead of a fixed heap region.

Q3. Where are objects and static variables stored in Java?

Objects (created with 'new') are always stored on the Heap, regardless of whether the reference variable is a local variable or an instance variable. Only the reference (pointer) to the object may live on the Stack (if it's a local variable) or inside another object on the Heap (if it's an instance field).

Static variables belong to the class, not to any instance, so they are stored in the Method Area / Metaspace, and only one copy exists per classloader regardless of how many objects are created.

Local primitive variables and references live on the Stack frame of the method call and are removed automatically once the method returns.

2. Java Streams API

Q4. What is the Stream API and how is it different from Collections?

A Stream is a sequence of elements that supports functional-style, declarative operations (map, filter, reduce, etc.) instead of external iteration. Streams don't store data - they operate on a source (Collection, array, I/O channel) and produce a result without modifying the source.

- Lazy evaluation - intermediate operations (map, filter) are not executed until a terminal operation (collect, forEach, reduce) is invoked.
- Can be used only once - once a terminal operation runs, the stream is consumed.
- Supports parallel execution via .parallelStream() using the ForkJoin common pool.
- Intermediate ops: map, filter, sorted, distinct, limit, skip, flatMap. Terminal ops: collect, forEach, reduce, count, anyMatch/allMatch.

Q5. Difference between map() and flatMap()?

map() transforms each element into exactly one new element (1-to-1), producing a Stream> if the mapper returns a stream. flatMap() transforms each element into a stream and then flattens all those streams into a single stream (1-to-many), which is essential when dealing with nested collections like List>.

Q6. Coding: Find the second highest salary of an Employee using Streams.

Sort employees by salary descending, skip the first (highest), then take the first remaining - or use distinct() first if duplicate salaries should be treated as one rank.

```
List<Employee> employees = ...;
Optional<Employee> secondHighest = employees.stream()
    .sorted(Comparator.comparing(Employee::getSalary).reversed())
    .distinct() // by equals()/hashCode() - or use a key extractor
    .skip(1)
    .findFirst();

// Cleaner: dedupe strictly by salary value
Optional<Double> secondHighestSalary = employees.stream()
    .map(Employee::getSalary)
    .distinct()
    .sorted(Comparator.reverseOrder())
    .skip(1)
    .findFirst();
```

Q7. Coding: Find the first non-repeating (unique) character in a string using Streams.

Build a frequency map with groupingBy + counting, then stream the characters in original order and find the first one whose count is 1 - preserving order requires a LinkedHashMap.

```
String input = "swiss";
Map<Character, Long> freq = input.chars()
    .mapToObj(c -> (char) c)
    .collect(Collectors.groupingBy(c -> c, LinkedHashMap::new, Collectors.counting()));

Character firstUnique = freq.entrySet().stream()
    .filter(e -> e.getValue() == 1)
    .map(Map.Entry::getKey)
    .findFirst()
    .orElse(null); // 'w'
```

Q8. Coding: Sort a list/array of integers or strings using Streams.

Use .sorted() for natural order or pass a Comparator for custom ordering; sorted() returns a new stream, the original list is untouched.

```
List<Integer> nums = List.of(5, 3, 8, 1, 9);
List<Integer> ascending = nums.stream().sorted().collect(Collectors.toList());
List<Integer> descending = nums.stream()
    .sorted(Comparator.reverseOrder())
    .collect(Collectors.toList());

// Sort strings by length, then alphabetically
List<String> words = List.of("pen","apple","cat","banana");
List<String> sorted = words.stream()
    .sorted(Comparator.comparing(String::length).thenComparing(Comparator.naturalOrder()))
    .collect(Collectors.toList());
```

Q9. What is the difference between Collectors.toList(), groupingBy() and partitioningBy()?

toList() simply gathers stream elements into a List. groupingBy() classifies elements into a Map keyed by a classifier function (like SQL GROUP BY), useful when there are more than two groups - e.g. grouping employees by department. partitioningBy() is a special case that always produces exactly two groups keyed by true/false based on a predicate - e.g. splitting employees into 'salary > 50000' and the rest.

Q10. What is the difference between findFirst() and findAny()?

findFirst() always returns the first element of the encounter order and is deterministic even in a parallel stream. findAny() returns any matching element and is allowed to return a different one on every run in a parallel stream because it's optimized for performance, not for order guarantees.

Q11. What is a Collector and can you write a custom one?

A Collector is a reduction operation defined by four functions - a supplier (creates the result container), an accumulator (adds an element to it), a combiner (merges two containers, used in parallel streams), and a finisher (final transformation). Collectors.of(...) lets you build a custom collector, though most use cases are covered by the built-in Collectors class (toList, toMap, joining, groupingBy, summarizingInt, etc.).

3. Java 8 Features

Q12. What are the major features introduced in Java 8?

- Lambda Expressions - concise syntax for implementing functional interfaces, e.g. (a, b) -> a + b.
- Functional Interfaces - interfaces with a single abstract method (Runnable, Comparator); java.util.function package added Function, Predicate, Supplier, Consumer, BiFunction, etc.
- Stream API - declarative, pipeline-based processing of collections.
- Default and Static methods in interfaces - allows adding new methods to interfaces without breaking existing implementations.
- Optional - a container object to avoid NullPointerException and force explicit handling of 'value may be absent'.
- New Date/Time API (java.time) - LocalDate, LocalDateTime, ZonedDateTime, Duration, Period - immutable and thread-safe, replacing the flawed java.util.Date/Calendar.
- Method References - shorthand for lambdas that just call an existing method, e.g. String::toUpperCase.
- Nashorn JavaScript Engine (removed later in Java 15).

Q13. What is a functional interface? Give examples.

A functional interface has exactly one abstract method (it can have any number of default/static methods) and can be marked with @FunctionalInterface for compile-time checking. It is the target type for a lambda expression.

- Runnable - run()
- Comparator - compare(T,T)
- Function - apply(T)
- Predicate - test(T)

- Supplier - get()
- Consumer - accept(T)

Q14. What is Optional and why was it introduced?

Optional is a wrapper that either contains a non-null value or is empty, forcing callers to explicitly handle the 'no value' case instead of silently risking a NullPointerException. Common methods: isPresent(), ifPresent(), orElse(), orElseGet(), orElseThrow(), map(), flatMap(). Best practice: use it as a method return type, not as a field or method parameter.

Q15. Default vs Static methods in interfaces?

A default method provides a body inside an interface and can be overridden by implementing classes; it was introduced so existing interfaces (like List) could add new methods (like forEach, stream) without breaking every class that already implements them. A static method belongs to the interface itself, cannot be overridden, and is called directly on the interface name, e.g. Comparator.comparing(...).

Q16. What is a lambda expression, and how is it different from an anonymous class?

A lambda expression is a concise way to provide the implementation of a functional interface. It is mainly used to pass behavior as data, especially with Streams and collection APIs.

- A lambda has no independent class name and is limited to one abstract method of the target functional interface; an anonymous class can implement a type and contain additional state and methods.
- Inside a lambda, this refers to the enclosing object; an anonymous class creates its own this context.
- Lambdas can capture local variables only when those variables are final or effectively final.
- Use lambdas for short behavior-focused logic; use an anonymous class when you need more state or a more involved implementation.

4. Comparable vs Comparator

Q17. Difference between Comparable and Comparator?

- Comparable is implemented by the class itself (compareTo method) and defines the object's single 'natural ordering', e.g. Employee implements Comparable.
- Comparator is a separate class/lambda (compare method) that defines external, custom ordering, and you can create many different Comparators for the same class without modifying it.
- Comparable -> java.lang, one sorting sequence per class. Comparator -> java.util, unlimited sorting sequences.
- Comparator supports composing rules: Comparator.comparing(Employee::getDept).thenComparing(Employee::getSalary, Comparator.reverseOrder()).

```
// Comparable
class Employee implements Comparable<Employee> {
    int salary;
    public int compareTo(Employee o) { return Integer.compare(this.salary, o.salary); }
}

// Comparator
List<Employee> list = ...;
list.sort(Comparator.comparing(Employee::getSalary).reversed()
    .thenComparing(Employee::getName));
```

5. Collections Framework, HashMap Internals & equals/hashCode

Q18. Explain the Java Collections Framework hierarchy.

- Collection -> List (ArrayList, LinkedList, Vector), Set (HashSet, LinkedHashSet, TreeSet), Queue (PriorityQueue, ArrayDeque).
- Map is a separate hierarchy (not a Collection) -> HashMap, LinkedHashMap, TreeMap, Hashtable, ConcurrentHashMap.
- List allows duplicates and maintains insertion order via index; Set disallows duplicates; Map stores key-value pairs with unique keys.

Q19. How does HashMap work internally?

A HashMap stores entries in an array of buckets. To insert a key, it computes hash(key.hashCode()) (an additional hash spreading function is applied to reduce clustering), then maps that hash to a bucket index using (n-1) & hash where n is the array length (always a power of two). The entry (key, value, hash, next) is placed in that bucket.

Default capacity is 16 with a load factor of 0.75; once size exceeds capacity * loadFactor, the internal array is resized (doubled) and all entries are rehashed - this is called resizing/rehashing.

Since Java 8, if a single bucket's linked list grows beyond 8 entries (and table capacity >= 64), that bucket is converted into a Red-Black Tree, changing worst-case lookup from O(n) to O(log n).

Q20. What is a HashMap collision and how is it handled?

A collision happens when two different keys produce the same bucket index (same hash & (n-1)), even if their hashCode() values differ, because multiple hash values can map to the same bucket.

- Java 7 and earlier: collisions are handled via chaining - a linked list of entries is stored per bucket; a colliding entry is appended to the list.
- Java 8+: still uses a linked list initially, but if a bucket's chain length exceeds TREEIFY_THRESHOLD (8) and the table has at least 64 buckets, the list is converted to a self-balancing Red-Black tree for that bucket, improving worst-case get/put from O(n) to O(log n).
- On lookup, once the correct bucket is found, equals() is used to distinguish between the colliding keys to find the exact match.

Q21. What is the relationship between equals() and hashCode()?

The general contract (from Object class) is: if two objects are equal according to equals(), they MUST have the same hashCode(). The reverse is not required - two unequal objects can share a hashCode (that's exactly a hash collision).

If you override equals() but not hashCode() (or vice versa), you break this contract, and hash-based collections (HashMap, HashSet, HashTable) will behave incorrectly - e.g. two 'equal' objects could end up in different buckets, so map.get(key) might return null even though an 'equal' key was inserted earlier.

Best practice: always override both together, and use the same fields in both methods (IDEs and Lombok's @EqualsAndHashCode automate this).

Q22. Difference between HashMap, LinkedHashMap and TreeMap?

- HashMap - no ordering guarantee, O(1) average get/put, allows one null key and multiple null values.
- LinkedHashMap - maintains insertion order (or access order if configured) using an internal doubly linked list; useful for building an LRU cache.
- TreeMap - maintains keys in sorted order (natural or via Comparator), backed by a Red-Black tree, O(log n) get/put; does not allow null keys.

Q23. How does HashSet work internally?

HashSet is backed by a HashMap. Each set element is stored as a HashMap key, while a shared dummy object is used as the value. HashSet therefore relies on hashCode() to locate a bucket and equals() to determine whether an equal element already exists.

- Average add(), remove(), and contains() are O(1), assuming a good hash distribution.
- Duplicate elements are rejected because HashMap keys must be unique.
- A mutable object used as a key can become effectively unreachable if fields used by hashCode()/equals() change after insertion.
- LinkedHashSet adds predictable insertion order, while TreeSet maintains sorted order.

Q24. How does ArrayList resize internally, and what is its time complexity?

ArrayList stores elements in a dynamically growing array. When the current capacity is insufficient, it allocates a larger array and copies the existing elements into it. The exact growth factor is an implementation detail; common OpenJDK implementations grow the array by roughly 50%.

- get(index) is O(1).
- Adding at the end is O(1) amortized because resizing happens only occasionally.
- Insertion or deletion in the middle is O(n) because elements may need to be shifted.
- Use ensureCapacity() when the approximate size is known to reduce repeated reallocations and copying.

6. LFU Cache Implementation

Q25. Design and implement an LFU (Least Frequently Used) Cache.

An LFU cache evicts the entry with the smallest access frequency when it's full; on a tie, the least recently used among those is evicted. An O(1) implementation uses three structures:

- keyToValFreq: Map - key -> (value, frequency)
- freqToKeys: Map> - frequency -> ordered set of keys at that frequency (LinkedHashSet keeps insertion/access order for tie-breaking)
- minFreq: tracks the current minimum frequency in the cache for O(1) eviction

```
class LFUCache {
    private final int capacity;
    private int minFreq;
    private final Map<Integer, int[]> keyToValFreq = new HashMap<>(); // key -> {value, freq}
    private final Map<Integer, LinkedHashSet<Integer>> freqToKeys = new HashMap<>();

    public LFUCache(int capacity) {
        this.capacity = capacity;
    }

    public int get(int key) {
        if (!keyToValFreq.containsKey(key)) return -1;
        bumpFrequency(key);
        return keyToValFreq.get(key)[0];
    }

    public void put(int key, int value) {
        if (capacity <= 0) return;
        if (keyToValFreq.containsKey(key)) {
            keyToValFreq.get(key)[0] = value;
            bumpFrequency(key);
            return;
        }
        if (keyToValFreq.size() >= capacity) {
            int evictKey = freqToKeys.get(minFreq).iterator().next();
            freqToKeys.get(minFreq).remove(evictKey);
            keyToValFreq.remove(evictKey);
        }
        keyToValFreq.put(key, new int[]{value, 1});
        freqToKeys.computeIfAbsent(1, k -> new LinkedHashSet<>()).add(key);
        minFreq = 1;
    }

    private void bumpFrequency(int key) {
        int freq = keyToValFreq.get(key)[1];
        freqToKeys.get(freq).remove(key);
        if (freqToKeys.get(freq).isEmpty()) {
            freqToKeys.remove(freq);
            if (minFreq == freq) minFreq++;
        }
        keyToValFreq.get(key)[1] = freq + 1;
        freqToKeys.computeIfAbsent(freq + 1, k -> new LinkedHashSet<>()).add(key);
    }
}
```

Q26. How would you implement an LRU Cache quickly (commonly asked alongside LFU)?

The simplest production-ready way is to extend LinkedHashMap with access-order enabled and override removeEldestEntry().

```
class LRUCache<K,V> extends LinkedHashMap<K,V> {
    private final int capacity;
    LRUCache(int capacity) {
        super(capacity, 0.75f, true); // true = access-order
        this.capacity = capacity;
    }
    @Override
    protected boolean removeEldestEntry(Map.Entry<K,V> eldest) {
        return size() > capacity;
    }
}
```

7. Fail-Fast vs Fail-Safe Collections

Q27. What is the difference between Fail-Fast and Fail-Safe collections?

- Fail-Fast: iterators operate directly on the original collection and throw ConcurrentModificationException immediately if the collection is structurally modified while iterating (except via the iterator's own remove()). Detected using an internal modCount counter compared on each next() call. Examples: ArrayList, HashMap, HashSet.
- Fail-Safe: iterators operate on a clone/snapshot of the collection (or use copy-on-write), so concurrent modification during iteration doesn't throw an exception - though the iterator may not reflect the latest changes. Examples: CopyOnWriteArrayList, ConcurrentHashMap.
- Trade-off: fail-safe avoids exceptions but costs extra memory (for the snapshot/copy) and can give you slightly stale data.

8. Thread-Safe Collections

Q28. What are the ways to make a collection thread-safe in Java?

- Legacy synchronized wrappers: Collections.synchronizedList/Map/Set(...) - wraps every method call with a single lock, so it's correct but has poor concurrency (whole collection is locked).
- Concurrent collections (java.util.concurrent) - purpose-built for high concurrency: ConcurrentHashMap (segmented/bucket-level locking), CopyOnWriteArrayList / CopyOnWriteArraySet (good for read-heavy, rarely-written lists), ConcurrentLinkedQueue (lock-free, non-blocking), BlockingQueue implementations (ArrayBlockingQueue, LinkedBlockingQueue) for producer-consumer patterns.
- Legacy classes that are inherently synchronized: Vector, Hashtable (rarely used today, slower than ConcurrentHashMap).

Q29. How does ConcurrentHashMap achieve thread safety without locking the whole map?

In Java 7, ConcurrentHashMap divided the map into a fixed number of segments, each with its own lock, so different threads could write to different segments concurrently. From Java 8 onward, segment locking was removed; instead it locks only the individual bucket (the head node of a bin) during a write using synchronized blocks combined with CAS (Compare-And-Swap) operations for the common case, giving much finer-grained concurrency. Reads are largely lock-free thanks to volatile reads on the table.

Q30. What is AtomicInteger, and when would you use it instead of synchronized?

AtomicInteger provides atomic operations on an int using non-blocking compare-and-set (CAS) techniques. It is useful for shared counters and simple state updates where a lock is unnecessary.

- incrementAndGet(), decrementAndGet(), compareAndSet(), and updateAndGet() perform atomic operations.
- It provides visibility and atomicity for the operations it exposes, but a sequence of multiple independent operations may still require additional coordination.
- Use synchronized or Lock when a larger critical section must be protected or multiple variables must change consistently.
- For a simple high-contention counter, AtomicInteger or LongAdder can be more appropriate than a synchronized block.

9. Coding Questions - Strings, Sorting, HashMap, LinkedList

Q31. Reverse a String / check if a String is a Palindrome.

```
String reversed = new StringBuilder(str).reverse().toString();

boolean isPalindrome(String s) {
    int i = 0, j = s.length() - 1;
    while (i < j) {
        if (s.charAt(i++) != s.charAt(j--)) return false;
    }
    return true;
}
```

Q32. Check if two Strings are Anagrams of each other.

```
boolean isAnagram(String a, String b) {
    if (a.length() != b.length()) return false;
    int[] freq = new int[256];
    for (char c : a.toCharArray()) freq[c]++;
    for (char c : b.toCharArray()) freq[c]--;
    for (int f : freq) if (f != 0) return false;
    return true;
}
```

Q33. Find duplicate elements in an array using HashMap/HashSet.

```
List<Integer> findDuplicates(int[] arr) {
    Set<Integer> seen = new HashSet<>();
    List<Integer> duplicates = new ArrayList<>();
    for (int n : arr) {
        if (!seen.add(n)) duplicates.add(n);
    }
    return duplicates;
}
```

Q34. Reverse a Singly Linked List (iterative).

```
Node reverse(Node head) {
    Node prev = null, curr = head;
    while (curr != null) {
        Node next = curr.next;
        curr.next = prev;
        prev = curr;
        curr = next;
    }
    return prev; // new head
}
```

Q35. Detect a Cycle in a Linked List (Floyd's Tortoise and Hare).

```
boolean hasCycle(Node head) {
    Node slow = head, fast = head;
    while (fast != null && fast.next != null) {
        slow = slow.next;
        fast = fast.next.next;
        if (slow == fast) return true;
    }
    return false;
}
```

Q36. Merge two sorted arrays / implement a merge sort by hand.

```
void merge(int[] a, int[] b, int[] result) {
    int i = 0, j = 0, k = 0;
    while (i < a.length && j < b.length) {
        result[k++] = (a[i] <= b[j]) ? a[i++] : b[j++];
    }
    while (i < a.length) result[k++] = a[i++];
    while (j < b.length) result[k++] = b[j++];
}
```

Q37. Coding: Solve the Two Sum problem efficiently.

Given an array and a target, return the indices of two numbers whose sum equals the target. The interview-friendly approach uses a HashMap to store previously seen values and their indices.

```
int[] twoSum(int[] nums, int target) {
    Map<Integer, Integer> seen = new HashMap<>();
    for (int i = 0; i < nums.length; i++) {
        int need = target - nums[i];
        if (seen.containsKey(need)) {
            return new int[]{seen.get(need), i};
        }
        seen.put(nums[i], i);
    }
    return new int[0];
}
```

Time complexity: O(n) average; space complexity: O(n). The interviewer may also ask for the brute-force O(n²) solution and why the HashMap approach improves it.

Q38. Coding: Implement Binary Search and explain its complexity.

Binary Search works on a sorted array by repeatedly comparing the target with the middle element and discarding half of the remaining search space.

```
int binarySearch(int[] arr, int target) {
    int left = 0, right = arr.length - 1;
    while (left <= right) {
        int mid = left + (right - left) / 2;
        if (arr[mid] == target) return mid;
        if (arr[mid] < target) left = mid + 1;
        else right = mid - 1;
    }
    return -1;
}
```

Time complexity is O(log n) and space complexity is O(1) for the iterative version. Always confirm that the input is sorted before applying Binary Search.

10. ArrayList vs LinkedList

Q39. Difference between ArrayList and LinkedList?

- Internal structure: ArrayList uses a dynamically resizable array; LinkedList uses a doubly linked list of nodes.
- Random access: ArrayList get(index) is O(1); LinkedList get(index) is O(n) since it must traverse from head or tail.
- Insertion/Deletion: at the beginning or middle, LinkedList is O(1) once you have the node reference (no shifting), while ArrayList is O(n) due to shifting elements. At the end, both are effectively O(1) amortized.
- Memory: ArrayList has lower per-element overhead; LinkedList needs extra memory per node for two pointers (prev/next).
- Use ArrayList by default (better cache locality, faster iteration); use LinkedList only when you need frequent insertions/deletions at both ends (it also implements Deque).

11. Cloning and Copy Constructor

Q40. What is Object cloning in Java? Shallow vs Deep copy?

Cloning creates a duplicate of an object using the clone() method inherited from Object, after implementing the marker interface Cloneable (calling clone() without implementing Cloneable throws CloneNotSupportedException).

- Shallow copy (default Object.clone()) - copies primitive fields directly, but for reference fields it copies only the reference, so the clone and original share the same nested objects.
- Deep copy - recursively clones nested/mutable objects too, so the clone is fully independent; typically implemented manually by overriding clone() to also clone each mutable field, or via serialization, or a copy constructor.

Q41. What is a Copy Constructor and how does it compare to clone()?

A copy constructor is a regular constructor that takes another object of the same class and copies its fields, e.g. public Employee(Employee other) { this.name = other.name; ... }. It is generally preferred over clone() because it doesn't require implementing Cloneable, doesn't throw a checked exception, gives full control over deep vs shallow copying, and works cleanly with final fields (clone() can't easily set final fields since it bypasses the constructor).

Q42. How do you design an immutable class in Java?

An immutable object cannot change its observable state after construction. A standard design makes the class final, fields private and final, initializes all state in the constructor, and exposes no mutating methods.

- Make the class final so it cannot be subclassed and altered through overriding.
- Make fields private and final; validate constructor arguments before publishing the object.
- Do not provide setters. Return defensive copies for mutable fields such as Date, arrays, or mutable collections.
- If a field is a collection, consider storing an unmodifiable copy rather than the caller's mutable reference.
- Immutable objects are naturally safe to share across threads because their state does not change.

12. Design Patterns - Singleton, Builder, Chain of Responsibility, Adapter

Q43. Implement a thread-safe Singleton (Bill Pugh / double-checked locking).

The Bill Pugh Singleton (static inner holder class) is the most recommended approach - it's lazy-loaded, thread-safe without explicit synchronization, and relies on the JVM's classloading guarantees.

```
public class Singleton {
    private Singleton() {}
    private static class Holder {
        private static final Singleton INSTANCE = new Singleton();
    }
    public static Singleton getInstance() {
        return Holder.INSTANCE;
    }
}

// Double-checked locking alternative
public class Singleton2 {
    private static volatile Singleton2 instance;
    private Singleton2() {}
    public static Singleton2 getInstance() {
        if (instance == null) {
            synchronized (Singleton2.class) {
                if (instance == null) {
                    instance = new Singleton2();
                }
            }
        }
        return instance;
    }
}
```

Q44. How can a Singleton be broken, and how do you prevent it?

- Reflection - can call the private constructor directly; prevent by throwing an exception inside the constructor if an instance already exists.
- Serialization - deserializing creates a new instance; prevent by implementing readResolve() to return the existing instance.
- Cloning - clone() can create a second instance; prevent by overriding clone() to throw CloneNotSupportedException.
- The safest approach that resists all three is an Enum-based Singleton, since enums are inherently serialization-safe and reflection-proof.

Q45. Implement the Builder pattern.

Builder is used to construct complex objects step by step, especially when a class has many optional fields, avoiding 'telescoping constructors'. It typically uses a static nested Builder class with fluent setters returning 'this', and a private constructor on the outer class that takes the Builder.

```
public class Employee {
    private final String name;
    private final int age;
    private final String department; // optional

    private Employee(Builder b) {
        this.name = b.name;
        this.age = b.age;
        this.department = b.department;
    }

    public static class Builder {
        private String name;
        private int age;
        private String department;
        public Builder name(String name) { this.name = name; return this; }
        public Builder age(int age) { this.age = age; return this; }
        public Builder department(String d) { this.department = d; return this; }
        public Employee build() { return new Employee(this); }
    }
}

// Usage
Employee e = new Employee.Builder().name("Ravi").age(28).department("Engineering").build();
```

Q46. Explain the Chain of Responsibility pattern with an example.

Chain of Responsibility passes a request along a chain of handler objects; each handler decides either to process the request or pass it to the next handler in the chain. This decouples the sender of a request from its receivers. It is used heavily in Java's Servlet Filter chains and Spring Security's FilterChain.

```
abstract class Approver {
    protected Approver next;
    void setNext(Approver next) { this.next = next; }
    abstract void approve(int amount);
}

class Manager extends Approver {
    void approve(int amount) {
        if (amount <= 1000) System.out.println("Manager approved");
        else if (next != null) next.approve(amount);
    }
}

class Director extends Approver {
    void approve(int amount) {
        if (amount <= 5000) System.out.println("Director approved");
        else if (next != null) next.approve(amount);
    }
}

// manager.setNext(director); manager.approve(3000);
```

Q47. Explain the Adapter pattern with an example.

Adapter converts the interface of one class into another interface that a client expects, letting incompatible interfaces work together - like a real-world power plug adapter. It's commonly used to integrate a legacy class or a third-party library whose interface doesn't match what your code expects.

```
interface MediaPlayer { void play(String fileName); }

class LegacyMp4Player { void playMp4(String fileName) { /* legacy logic */ } }

class MediaAdapter implements MediaPlayer {
    private LegacyMp4Player mp4Player = new LegacyMp4Player();
    public void play(String fileName) {
        mp4Player.playMp4(fileName); // adapts the call
    }
}
```

Q48. Explain the Factory Method pattern and when it is useful.

Factory Method defines a method for creating an object while allowing subclasses or implementations to decide which concrete type is instantiated. It separates object creation from the code that uses the object.

- Useful when creation logic is conditional or likely to change.
- The client depends on an abstraction rather than concrete implementations.
- It improves extensibility because new implementations can be introduced without changing the main business logic.
- Do not introduce a factory only for trivial object construction; the pattern is most useful when creation has variability or policy.

```
interface Notification { void send(String message); }

class EmailNotification implements Notification {
    public void send(String message) { /* send email */ }
}

class NotificationFactory {
    static Notification create(String type) {
        if ("email".equalsIgnoreCase(type)) return new EmailNotification();
        throw new IllegalArgumentException("Unsupported type");
    }
}
```

Q49. Explain the Strategy pattern with a Java example.

Strategy defines a family of interchangeable algorithms behind a common interface. The client can select or change the algorithm without changing the code that uses it.

- Each strategy encapsulates one algorithm or business rule.
- The context depends on the strategy interface, reducing large if-else or switch blocks.
- Strategies can be selected at runtime based on configuration, request type, or business conditions.
- It is especially useful when multiple implementations follow the same contract but differ in behavior.

```
interface PaymentStrategy {
    void pay(double amount);
}

class CardPayment implements PaymentStrategy {
    public void pay(double amount) { /* card payment */ }
}

class PaymentService {
    private final PaymentStrategy strategy;
    PaymentService(PaymentStrategy strategy) { this.strategy = strategy; }
    void checkout(double amount) { strategy.pay(amount); }
}
```

13. Spring Boot - Annotations, REST, Exception Handling & @Transactional

Q50. Explain the commonly used Spring Boot annotations.

- @SpringBootApplication - combines @Configuration, @EnableAutoConfiguration and @ComponentScan.
- @RestController = @Controller + @ResponseBody, returns data (JSON/XML) directly instead of a view name.
- @RequestMapping / @GetMapping / @PostMapping / @PutMapping / @DeleteMapping - map HTTP requests to handler methods.
- @Autowired - injects a Spring-managed bean by type (constructor injection is now the recommended style over field injection).
- @Component / @Service / @Repository - stereotype annotations that register a class as a Spring bean; @Repository additionally translates persistence exceptions into Spring's DataAccessException hierarchy.
- @Qualifier - resolves ambiguity when multiple beans of the same type exist.
- @Value - injects a property value from application.properties/yml.
- @ConfigurationProperties - binds a whole group of properties to a POJO.
- @RequestParam / @PathVariable / @RequestBody / @RequestHeader - extract data from the incoming HTTP request.
- @Transactional - wraps a method in a database transaction.
- @ExceptionHandler / @ControllerAdvice - centralized exception handling.

Q51. How do you design REST APIs properly (best practices)?

- Use nouns for resources, not verbs: /employees not /getEmployees.
- Use correct HTTP methods: GET (read), POST (create), PUT (full update), PATCH (partial update), DELETE (remove).
- Use proper status codes: 200 OK, 201 Created, 204 No Content, 400 Bad Request, 401 Unauthorized, 403 Forbidden, 404 Not Found, 409 Conflict, 500 Internal Server Error.
- Version your APIs (/api/v1/...), support pagination/filtering/sorting via query params, and keep responses consistent using a common response wrapper/DTO.

Q52. How is exception handling done in Spring Boot?

Spring Boot centralizes exception handling using @ControllerAdvice combined with @ExceptionHandler, so you don't need try-catch blocks scattered across every controller. A global handler class intercepts exceptions thrown by any controller and converts them into a consistent error response (often a custom ErrorResponse DTO with status, message, timestamp).

```
@ControllerAdvice
public class GlobalExceptionHandler {
    @ExceptionHandler(ResourceNotFoundException.class)
    public ResponseEntity<ErrorResponse> handleNotFound(ResourceNotFoundException ex) {
        ErrorResponse err = new ErrorResponse(HttpStatus.NOT_FOUND.value(), ex.getMessage());
        return new ResponseEntity<>(err, HttpStatus.NOT_FOUND);
    }

    @ExceptionHandler(Exception.class)
    public ResponseEntity<ErrorResponse> handleGeneric(Exception ex) {
        ErrorResponse err = new ErrorResponse(500, "Something went wrong");
        return new ResponseEntity<>(err, HttpStatus.INTERNAL_SERVER_ERROR);
    }
}
```

Q53. Explain @Transactional in depth - propagation and isolation levels.

@Transactional tells Spring's AOP proxy to wrap a method call in a database transaction, automatically committing on success or rolling back on a RuntimeException (checked exceptions do NOT trigger rollback by default unless configured with rollbackFor).

- Propagation.REQUIRED (default) - joins an existing transaction, or creates a new one if none exists.

- Propagation.REQUIRES_NEW - always suspends any existing transaction and starts a brand-new independent one.
- Propagation.NESTED - runs within a nested transaction (savepoint) inside the existing one; can roll back independently.
- Propagation.SUPPORTS / NOT_SUPPORTED / MANDATORY / NEVER - other less common combinations.
- Isolation levels: READ_UNCOMMITTED, READ_COMMITTED (most common default in Postgres/Oracle), REPEATABLE_READ (MySQL InnoDB default), SERIALIZABLE (strictest, prevents phantom reads but hurts concurrency).
- Common pitfall: calling a @Transactional method from another method in the SAME class doesn't go through the Spring proxy, so the transaction is silently ignored - it must be called from a different bean/class.

Q54. What are IoC and Dependency Injection in Spring?

Inversion of Control means the framework manages object creation and lifecycle instead of application code manually constructing every dependency. Dependency Injection is the mechanism Spring uses to provide those dependencies to a bean.

- Constructor injection is generally preferred because dependencies are explicit, can be final, and are available when the object is created.
- Spring's ApplicationContext creates and wires beans according to configuration and component scanning.
- DI reduces tight coupling and makes classes easier to test by allowing dependencies to be replaced with mocks or alternate implementations.
- @Component, @Service, @Repository, and @Controller are common ways to register application components as Spring beans.

Q55. What are Spring Bean scopes and what is the bean lifecycle?

A Spring bean has a scope that controls how many instances Spring creates and where they are reused. The default scope is singleton, meaning one bean instance per Spring ApplicationContext.

- Singleton - one instance per container; prototype - a new instance when requested; request/session/application/websocket scopes are available in web applications.
- Typical lifecycle: instantiate bean -> inject dependencies -> initialization callbacks such as @PostConstruct -> bean is ready for use -> destruction callback such as @PreDestroy for managed scopes.
- Prototype beans are created by Spring but their destruction lifecycle is generally not managed by the container after creation.
- Avoid storing request-specific mutable state inside singleton beans because the same instance may serve many concurrent requests.

Q56. What is Spring Boot auto-configuration, and how does it work?

Spring Boot auto-configuration attempts to configure commonly used beans based on the application's classpath, properties, and existing beans. It reduces boilerplate while still allowing developers to override the defaults.

- @SpringBootApplication includes @EnableAutoConfiguration, which activates auto-configuration.
- Auto-configurations use conditional checks such as presence of a class, missing bean, or property value before creating configuration.
- If the application defines its own bean, many auto-configurations back off to avoid overriding explicit application configuration.
- Actuator's conditions/configuration reports can help diagnose why an auto-configuration was or was not applied.

Q57. How do Spring Profiles and externalized configuration work?

Profiles allow different configuration or beans to be activated for environments such as development, testing, and production. Externalized configuration keeps environment-specific values outside the application code.

- Use application.yml/properties plus profile-specific files such as application-dev.yml and application-prod.yml.
- Activate a profile with spring.profiles.active or an equivalent deployment/environment setting.
- Use @Profile to conditionally register beans and @ConfigurationProperties for structured configuration binding.
- Keep secrets out of source control; inject them through environment variables, secret managers, or deployment configuration.

14. JPA, Hibernate & Database Locking

Q58. Difference between JPA and Hibernate?

JPA (Java Persistence API) is just a specification/interface (a set of annotations and interfaces like EntityManager, @Entity) that defines how ORM should work in Java - it has no implementation of its own. Hibernate is one of several concrete implementations of the JPA spec (others include EclipseLink, OpenJPA), and also provides extra Hibernate-specific features beyond the JPA spec (like @NaturalId, custom types, and a richer Criteria API).

Q59. Explain First-level vs Second-level cache in Hibernate.

- First-level cache - tied to the Hibernate Session/EntityManager, enabled by default, not shared across sessions; cleared when the session closes.
- Second-level cache - optional, configured at the SessionFactory level, shared across sessions (often backed by EhCache, Caffeine, or Redis), useful for read-mostly reference data.

Q60. What is the N+1 select problem and how do you fix it?

N+1 happens when fetching a list of N parent entities triggers 1 query for the parents, then N additional queries (one per parent) to lazily fetch each related child collection - devastating for performance at scale.

- Use JOIN FETCH in JPQL to eagerly fetch the association in a single query.
- Use @EntityGraph to declaratively specify which associations to fetch eagerly for a specific query.
- Enable batch fetching (hibernate.default_batch_fetch_size) so Hibernate fetches related entities in batches instead of one-by-one.

Q61. Explain Optimistic vs Pessimistic Locking in JPA.

- Optimistic Locking - assumes conflicts are rare; each entity has a @Version column (int/timestamp). On update, Hibernate checks the version hasn't changed since it was read; if it has, an OptimisticLockException is thrown. No DB-level lock is held, so it scales better for read-heavy systems.
- Pessimistic Locking - assumes conflicts are likely; acquires an actual database row lock (SELECT ... FOR UPDATE) at read time via LockModeType.PESSIMISTIC_WRITE/READ, blocking other transactions from modifying (or even reading, depending on mode) that row until the transaction completes.
- Use optimistic locking for high-concurrency, low-conflict scenarios (e.g. e-commerce catalog updates); use pessimistic locking when correctness under heavy contention is critical (e.g. seat booking, bank balance updates).

Q62. Difference between FetchType.LAZY and FetchType.EAGER?

LAZY defers loading an association until it's actually accessed (returns a proxy), reducing unnecessary queries but risking a LazyInitializationException if accessed outside an open Hibernate session. EAGER loads the association immediately along with the parent entity, which is simpler but can cause performance issues and accidental N+1 queries if overused. Best practice: default everything to LAZY and fetch eagerly only when explicitly needed via JOIN FETCH or @EntityGraph.

Q63. Explain the JPA entity lifecycle states.

A JPA entity moves through well-defined lifecycle states as it is created, persisted, queried, detached, and removed. Understanding these states is important for predicting when SQL is generated and when changes are tracked.

- Transient - a new object not associated with a persistence context.
- Managed/Persistent - attached to the EntityManager persistence context; changes can be detected automatically through dirty checking.
- Detached - previously managed but no longer attached, for example after clear(), detach(), or closing the persistence context.
- Removed - marked for deletion from the database when the transaction is flushed/committed.
- persist() makes a new entity managed; merge() copies detached state into a managed instance rather than simply re-attaching the same object.

Q64. What is the difference between cascade types and orphanRemoval in JPA?

Cascade controls which persistence operations are propagated from a parent entity to related entities. orphanRemoval controls whether a child removed from an association is automatically deleted from the database.

- CascadeType.PERSIST, MERGE, REMOVE, REFRESH, DETACH, and ALL control propagation of corresponding operations.
- orphanRemoval=true is commonly used for privately owned child entities where removing the child from the parent's collection should delete the child row.
- Cascade REMOVE can delete children when the parent itself is deleted; orphanRemoval is about the child becoming an orphan from the relationship.
- Use these options carefully because cascading REMOVE across a large graph can cause unexpected deletes.

15. Asynchronous Processing in Spring (@Async)

Q65. How does @Async work in Spring, and what are its pitfalls?

@Async makes a method execute in a separate thread (from a configured TaskExecutor) instead of blocking the caller's thread; the method must be public and called from a different bean (proxy limitation, same as @Transactional). Enable it with @EnableAsync on a configuration class.

- Return types: void (fire-and-forget), or Future/CompletableFuture if the caller needs the eventual result.
- Always configure a custom ThreadPoolTaskExecutor bean - the default SimpleAsyncTaskExecutor creates a new thread per call with no pooling/limits, which can exhaust resources under load.
- Exceptions thrown inside a void @Async method are swallowed unless you configure an AsyncUncaughtExceptionHandler.
- @Async is proxy-based AOP, so self-invocation (calling the async method from within the same class) does NOT go through the proxy and runs synchronously.

```
@Async
public CompletableFuture<String> processOrder(Order order) {
    // long-running work
    return CompletableFuture.completedFuture("done");
}
```

16. ExecutorService - Deep Dive

Q66. What is ExecutorService and why use it over creating raw Threads?

Creating a new Thread per task is expensive and unbounded (can exhaust memory/CPU under load). ExecutorService, from java.util.concurrent, manages a pool of reusable worker threads, decoupling task submission from execution, and provides lifecycle control (shutdown, shutdownNow, awaitTermination) and result handling via Future.

Q67. What are the different types of thread pools in Executors?

- newFixedThreadPool(n) - fixed number of threads; extra tasks wait in an unbounded queue; good for a known, stable workload.

- newCachedThreadPool() - creates new threads as needed and reuses idle ones (idle threads killed after 60s); good for many short-lived async tasks but risky under heavy load (unbounded thread creation).
- newSingleThreadExecutor() - a single worker thread, tasks execute sequentially in submission order.
- newScheduledThreadPool(n) - supports delayed and periodic task execution (scheduleAtFixedRate, scheduleWithFixedDelay).
- In production, it's now recommended to build a ThreadPoolExecutor explicitly (core size, max size, queue capacity, RejectedExecutionHandler) rather than the Executors factory methods, since some factory methods use unbounded queues that can cause OutOfMemoryError.

Q68. Explain the core parameters of ThreadPoolExecutor.

- corePoolSize - minimum number of threads kept alive even when idle.
- maximumPoolSize - max threads allowed when the queue is full.
- keepAliveTime - how long idle threads beyond corePoolSize wait before terminating.
- workQueue - holds tasks waiting for a free thread (e.g. LinkedBlockingQueue, ArrayBlockingQueue, SynchronousQueue).
- RejectedExecutionHandler - strategy when both the pool and queue are full: AbortPolicy (throws exception, default), CallerRunsPolicy (caller thread runs the task, provides natural backpressure), DiscardPolicy, DiscardOldestPolicy.

Q69. Difference between Future and CompletableFuture?

Future represents the result of an asynchronous computation but only supports a blocking get() to retrieve the result - it has no way to attach a callback, combine with other futures, or handle exceptions declaratively. CompletableFuture (Java 8+) implements Future but adds a rich, non-blocking, functional API: thenApply/thenAccept for transforming results, thenCompose for chaining dependent async calls, thenCombine for combining two independent futures, and exceptionally/handle for error handling - enabling full async pipelines without blocking any thread.

Q70. How do you properly shut down an ExecutorService?

```
executor.shutdown(); // stop accepting new tasks, let existing ones finish
try {
    if (!executor.awaitTermination(30, TimeUnit.SECONDS)) {
        executor.shutdownNow(); // force-cancel remaining tasks
    }
} catch (InterruptedException e) {
    executor.shutdownNow();
    Thread.currentThread().interrupt();
}
```

17. Microservices - Types, Communication, Circuit Breaker & JWT

Q71. What are Microservices and how are they different from Monolithic architecture?

A microservices architecture splits an application into small, independently deployable services, each owning its own data store and business capability, communicating over the network (typically HTTP/REST or messaging). A monolith, in contrast, is a single deployable unit with a shared codebase and database. Microservices give independent scaling, technology flexibility, and fault isolation, but add complexity around network calls, data consistency, and operational overhead (deployment, monitoring, tracing).

Q72. How do microservices communicate with each other?

- Synchronous - REST over HTTP (simplest, but couples availability - if the callee is down, the call fails), or gRPC (binary, faster, uses Protocol Buffers, supports streaming).
- Asynchronous - message brokers like Kafka or RabbitMQ, where a service publishes an event and one or more services consume it independently; this decouples services in time and improves resilience.
- Service discovery (Eureka, Consul) lets services find each other's network location dynamically instead of hardcoding hostnames.
- An API Gateway (Spring Cloud Gateway, Netflix Zuul, Kong) sits at the edge, routing external client requests to the right internal service, and can also handle auth, rate limiting, and request aggregation.

Q73. What is the Circuit Breaker pattern and how is it implemented (Resilience4j)?

Circuit Breaker prevents a service from repeatedly calling a downstream dependency that is already failing, avoiding cascading failures and wasted resources/threads. It has three states:

- CLOSED - requests flow normally; failures are counted.
- OPEN - once the failure rate crosses a threshold, the circuit 'trips' and calls fail immediately (fast-fail) without hitting the downstream service, for a configured wait duration.
- HALF_OPEN - after the wait duration, a limited number of trial requests are allowed through; if they succeed, the circuit closes again, otherwise it re-opens.

```
@CircuitBreaker(name = "inventoryService", fallbackMethod = "fallback")
public String checkInventory(String sku) {
    return restTemplate.getForObject("http://inventory-service/check/" + sku, String.class);
}

public String fallback(String sku, Throwable t) {
    return "Inventory service unavailable, please try later";
}
```

Q74. What is JWT and how is it used for authentication in microservices?

JWT (JSON Web Token) is a compact, self-contained, digitally signed token with three Base64-encoded parts: Header (algorithm), Payload (claims - user id, roles, expiry), and Signature (verifies the token hasn't been tampered with, using HMAC or RSA).

- Flow: user logs in -> Auth service validates credentials and issues a JWT -> client sends the JWT in the Authorization: Bearer header on every subsequent request -> each microservice independently validates the signature and reads claims, without calling back to the Auth service or a shared session store.
- This makes JWT ideal for stateless microservices - no server-side session storage is needed, and any service that trusts the signing key can validate the token.
- Access tokens are short-lived; a longer-lived Refresh token is used to obtain a new access token without re-authenticating.
- Care needed: tokens can't be easily revoked before expiry (mitigated with short expiry + a token blacklist/Redis check, or opaque tokens validated against an Auth server).

Q75. What are the different types/patterns of microservices decomposition?

- Decomposition by Business Capability - each service maps to a business function (Order Service, Payment Service, Inventory Service).
- Decomposition by Subdomain (DDD) - services align with Domain-Driven Design bounded contexts.
- Strangler Fig pattern - incrementally migrating a monolith to microservices by routing specific functionality to new services while the rest still hits the monolith.
- Saga pattern - manages distributed transactions across services using a sequence of local transactions with compensating actions on failure (choreography-based via events, or orchestration-based via a central coordinator).
- API Composition / CQRS - for queries that need data from multiple services, either aggregate at the gateway/API composer, or maintain a separate read-optimized view updated via events (CQRS).

Q76. How is Distributed Tracing / Observability handled in microservices?

Since a single user request can span many services, tools like Spring Cloud Sleuth (or Micrometer Tracing in newer Spring Boot) attach a unique Trace ID and Span ID to each request, propagated across service boundaries via HTTP headers. These traces are collected and visualized in tools like Zipkin or Jaeger, alongside centralized logging (ELK stack) and metrics (Prometheus + Grafana) for full observability.

Q77. How do you maintain data consistency across microservices without a distributed transaction?

A common approach is eventual consistency: each service owns its data and local transaction, while events communicate state changes between services. The Saga pattern coordinates the sequence of local transactions and compensating actions when a later step fails.

- Choreography uses events so services react independently; orchestration uses a coordinator that tells participants what to do.
- The Outbox pattern can reliably publish an event by storing the business change and the event record in the same local database transaction, then publishing the outbox event asynchronously.
- Consumers should be idempotent because messages can be delivered more than once.
- Retries, timeouts, dead-letter handling, and reconciliation jobs are often needed because network and service failures are normal in distributed systems.

18. Spring Security

Q78. Explain the Spring Security filter chain and authentication flow.

Every incoming HTTP request passes through a chain of Servlet Filters before reaching the DispatcherServlet. Key filters include UsernamePasswordAuthenticationFilter (form login), BasicAuthenticationFilter, and a custom JwtAuthenticationFilter for token-based auth. The AuthenticationManager delegates to one or more AuthenticationProvider implementations to actually verify credentials (e.g. checking a hashed password via a UserDetailsService), and on success populates the SecurityContext with an Authentication object that the rest of the request can rely on.

Q79. How do you secure a REST API with JWT in Spring Security?

Since REST APIs are stateless, session-based login (JSESSIONID cookies) is replaced with a custom filter that extracts the JWT from the Authorization header, validates its signature/expiry, loads the user's authorities from the token claims, and manually sets the Authentication into the SecurityContextHolder - all before the request reaches the controller. Session creation is disabled with SessionCreationPolicy.STATELESS.

```
http.csrf(csrf -> csrf.disable())
    .sessionManagement(sm -> sm.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
    .authorizeHttpRequests(auth -> auth
        .requestMatchers("/auth/**").permitAll()
        .anyRequest().authenticated())
    .addFilterBefore(jwtAuthFilter, UsernamePasswordAuthenticationFilter.class);
```

Q80. Difference between Authentication and Authorization?

Authentication answers 'who are you?' - verifying identity (username/password, JWT, OAuth token). Authorization answers 'what are you allowed to do?' - checking whether the authenticated user has permission for a specific action or resource, typically via roles/authorities (@PreAuthorize("hasRole('ADMIN')")).

Q81. What is CSRF and why is it usually disabled for stateless REST APIs?

CSRF (Cross-Site Request Forgery) tricks an authenticated user's browser into submitting an unwanted request using their existing session cookie. Spring Security's CSRF protection is relevant for cookie/session-based browser apps. For stateless REST APIs authenticated via a bearer token in a header (not an automatically-sent

cookie), CSRF isn't applicable in the same way since the token must be explicitly attached by the client, so it's commonly disabled - though care is still needed if tokens are ever stored in cookies.

19. AWS Services (commonly asked for Java/Spring Boot roles)

Q82. Which AWS services are commonly asked about for a Java/Spring Boot backend role?

- EC2 - virtual servers to host the application; key concepts: AMIs, instance types, Auto Scaling Groups, Security Groups (instance-level firewall).
- S3 - object storage for files/static assets/backups; durability via versioning, lifecycle policies to move data to Glacier for cost savings.
- RDS - managed relational databases (MySQL/Postgres/Oracle); handles backups, patching, Multi-AZ failover, read replicas for scaling reads.
- Elastic Load Balancer (ALB/NLB) - distributes incoming traffic across multiple EC2 instances/containers for high availability.
- Lambda - serverless functions that run code in response to events (API Gateway request, S3 upload, SQS message) without managing servers, billed per invocation/duration.
- SQS - managed message queue for decoupling services asynchronously (standard = at-least-once/best-effort ordering, FIFO = exactly-once processing and strict ordering).
- SNS - pub/sub notification service, often paired with SQS (fan-out pattern: one SNS topic delivering to multiple SQS queues).
- IAM - manages users, roles, and fine-grained permissions/policies for who can access which AWS resource.
- CloudWatch - metrics, logs, and alarms for monitoring application and infrastructure health.
- ECS / EKS - container orchestration for running Dockerized Spring Boot apps (ECS is AWS-native; EKS is managed Kubernetes).
- API Gateway - manages, secures, and throttles API endpoints, often fronting Lambda functions.

Q83. How would you deploy a Spring Boot application on AWS (typical setup)?

A common pattern: package the app as a Docker image, push it to ECR (Elastic Container Registry), deploy it on ECS Fargate (serverless containers, no EC2 management) or EKS, put it behind an Application Load Balancer, use RDS for the database, store secrets in AWS Secrets Manager or Parameter Store, and monitor with CloudWatch. For simpler use cases, Elastic Beanstalk offers a more managed, opinionated deployment path directly from a JAR file.

20. Apache Kafka - Topics, Producers, Consumer Groups & Idempotency

Q84. Explain Kafka's core architecture - Topics, Partitions, Brokers.

A Kafka Topic is a named, append-only log of messages, split into one or more Partitions for parallelism and scalability. Each partition is an ordered, immutable sequence of messages, and order is guaranteed only within a partition, not across the whole topic. A Broker is a single Kafka server; a cluster consists of multiple brokers, and each partition is replicated across brokers for fault tolerance (one broker holds the 'leader' replica that handles all reads/writes, others hold 'follower' replicas that stay in sync).

Q85. How do Kafka Producers and Consumer Groups work?

- A Producer sends messages to a topic; it can specify a key, and Kafka uses the key's hash to consistently route messages with the same key to the same partition, preserving order for that key (e.g. all events for the same order ID land on one partition).
- A Consumer Group is a set of consumers that jointly consume a topic - Kafka guarantees each partition is consumed by only ONE consumer within a group at a time, enabling horizontal scaling (add more consumers, up to the number of partitions, to increase throughput).
- Multiple consumer groups can independently consume the same topic in full - the partition-to-single-consumer rule only applies within a group.
- Consumers track their position via an 'offset' per partition, periodically committed back to Kafka (in an internal __consumer_offsets topic), so a restarted consumer resumes from where it left off.

Q86. What is Kafka Idempotency and how do you avoid duplicate message processing?

- Producer idempotency - enabling enable.idempotence=true assigns each producer a unique Producer ID and a sequence number per partition, so Kafka brokers can detect and discard duplicate retries of the same message (e.g. caused by a network timeout after the write actually succeeded), guaranteeing exactly-once delivery at the producer-to-broker level.
- Consumer-side idempotency - since Kafka's default delivery semantics are 'at-least-once' (a consumer might reprocess a message after a crash/rebalance before committing its offset), the consuming application itself must be designed to be idempotent - e.g. using a unique message/business key to check 'have I already processed this?' before applying it (via a DB unique constraint, a processed-IDs table, or idempotent upsert operations) so re-delivery doesn't cause duplicate side effects.
- For true exactly-once processing end-to-end, Kafka also supports Transactions (transactional.id) so a consume-process-produce cycle can be committed atomically across topics.

Q87. Difference between Kafka and traditional Message Queues like RabbitMQ?

- Kafka retains messages for a configured retention period (or forever) even after consumption, allowing multiple independent consumers to replay history; RabbitMQ typically removes a message once acknowledged/consumed.
- Kafka is built for very high throughput, ordered log-based streaming (event sourcing, log aggregation, stream processing); RabbitMQ is a more traditional message broker with richer routing (exchanges, topic/fanout/direct bindings) suited to complex routing logic and lower-latency task queues.
- Kafka scales via partitions and consumer groups; RabbitMQ scales via more queues/consumers and clustering.

Q88. What are Kafka delivery semantics, acknowledgments, and the trade-offs?

Kafka systems are commonly discussed in terms of at-most-once, at-least-once, and exactly-once processing semantics. Producer acknowledgment settings and consumer offset management strongly influence the resulting guarantees.

- acks=0: the producer does not wait for a broker acknowledgment; lowest durability guarantee and potentially lower latency.
- acks=1: the leader acknowledges the record after writing it locally; a leader failure before replication can still cause data loss.
- acks=all (or -1): the leader waits for the in-sync replicas required by min.insync.replicas, giving stronger durability.
- At-least-once is common for consumers: process the record and commit the offset carefully, while making processing idempotent to tolerate retries.
- Exactly-once semantics are possible for supported Kafka consume-process-produce workflows using transactions, but end-to-end external side effects still require careful design.

21. General / Frequently Asked Core Java Questions

Q89. What is the difference between == and equals()?

== compares references for objects (checks if both point to the same memory location) or actual values for primitives. equals() (when properly overridden, e.g. in String, wrapper classes, or your own classes) compares logical/content equality. Two different String objects with the same characters are '==' false but '.equals()' true, unless they come from the String pool via literals.

Q90. What is the difference between String, StringBuilder and StringBuffer?

- String - immutable; every modification creates a new object; safe to share across threads but inefficient for heavy concatenation in a loop.
- StringBuilder - mutable, NOT thread-safe, faster; use for single-threaded string building (e.g. inside a loop).
- StringBuffer - mutable and thread-safe (synchronized methods), but slower than StringBuilder due to synchronization overhead; rarely needed today since most string-building happens on a single thread.

Q91. Why is String immutable in Java?

- Security - Strings are used for things like class names, file paths, network connections, and DB URLs; immutability prevents them from being altered after validation.
- String pool / caching - immutability lets the JVM safely reuse the same String object for identical literals, saving memory.
- Thread safety - an immutable object can be freely shared across threads without synchronization.
- HashCode caching - since content can't change, String caches its hashCode after first computation, making it a very efficient HashMap key.

Q92. Difference between abstract class and interface?

- An abstract class can have both abstract and concrete methods, constructors, instance state/fields, and any access modifier; a class can extend only ONE abstract class.
- An interface (pre-Java 8) could only declare abstract methods and constants; since Java 8 it can also have default and static methods, and since Java 9, private methods - but still no instance state (only implicitly public static final fields). A class can implement MULTIPLE interfaces.
- Use an abstract class for a strong 'is-a' relationship with shared code/state; use an interface to define a contract/capability that unrelated classes can all implement (e.g. Comparable, Serializable).

Q93. Explain method overloading vs overriding.

- Overloading - same method name, different parameter list, within the SAME class; resolved at compile time (static/early binding).
- Overriding - subclass provides a specific implementation of a method already defined in its superclass, with the SAME signature; resolved at runtime based on the actual object type (dynamic/late binding), enabling polymorphism.

Q94. What is the difference between checked and unchecked exceptions?

Checked exceptions (subclasses of Exception, excluding RuntimeException) must be either caught or declared in the method signature with 'throws' - the compiler enforces handling (e.g. IOException, SQLException), typically representing recoverable conditions external to the program. Unchecked exceptions (subclasses of RuntimeException, like NullPointerException, ArrayIndexOutOfBoundsException) are not required to be declared or caught, and typically represent programming bugs.

Q95. What is the 'volatile' keyword and how is it different from 'synchronized'?

volatile guarantees visibility - any write to a volatile variable by one thread is immediately visible to all other threads (it prevents CPU/JIT caching of the variable in a thread-local register), but it does NOT provide atomicity for compound operations (like i++). synchronized provides both visibility AND atomicity/mutual exclusion by acquiring a lock, ensuring only one thread executes the critical section at a time, but at a higher performance cost. Use volatile for simple flags (e.g. a boolean 'running' flag checked by multiple threads); use synchronized (or java.util.concurrent.atomic classes) when a compound read-modify-write needs to be atomic.

Q96. What are Java Memory Model guarantees around the 'final' keyword?

A final field, once fully initialized in the constructor, is guaranteed to be visible to all threads that see a reference to the object, without needing extra synchronization - this is why immutable objects (with all-final fields) are inherently thread-safe for reads, as long as the object reference itself isn't published unsafely (e.g. leaking 'this' before construction finishes).

Q97. What is the difference between throw and throws?

throw is used inside a method body to actually raise/instantiate a specific exception at runtime (throw new IllegalArgumentException("bad input")). throws is used in a method signature to declare that the method might propagate one or more checked exceptions to its caller, who must then handle or further declare them.

Q98. What is Garbage Collection and how do the common GC algorithms differ?

- Serial GC - single-threaded, stop-the-world; best for small applications/single-CPU environments.
- Parallel GC - multiple threads for young-gen collection, still stop-the-world but faster than Serial; throughput-oriented (default in older JDKs).
- G1 (Garbage First) GC - divides heap into regions, prioritizes collecting regions with the most garbage first, aims for predictable pause times; default GC since Java 9.
- ZGC / Shenandoah - modern low-latency collectors designed for sub-millisecond pause times even on very large (multi-GB/TB) heaps.

Q99. What is the difference between process and thread, and what causes a deadlock?

A process is an independent program with its own memory space; a thread is a lightweight unit of execution within a process, sharing the same memory/heap with other threads of that process, which is what makes inter-thread communication fast but also introduces synchronization concerns.

A deadlock occurs when two or more threads are each waiting for a lock held by the other, so neither can proceed - classic example: Thread A holds Lock1 and waits for Lock2, while Thread B holds Lock2 and waits for Lock1. Prevention: always acquire multiple locks in a consistent global order, use tryLock() with a timeout, or minimize the scope/number of locks held simultaneously.

Q100. Is Java pass-by-value or pass-by-reference?

Java is always pass-by-value. For object arguments, the value being copied is the object reference, so a method can mutate the referenced object's state but cannot replace the caller's reference itself.

```
void change(Employee e) {
    e.setName("Updated");       // caller sees the state change
    e = new Employee("New");    // caller's reference is unchanged
}
```

- Primitive arguments copy the primitive value.
- Object arguments copy the reference value, not the object itself.
- Reassigning the parameter does not change the caller's variable because the parameter is a separate copy of the reference.